

STUDENT ID: 11605966

The Maven Rail data originates from a site called Maven Analytics that conducts challenges for data scientists. This data was added on 15th May'24 and includes mock data about UK's National Rail from January to April 2024. It has 31,653 records, 18 fields, and a public domain license (i.e. it's free to use).

Field	Description	Example
Payment Method	Payment method used to purchase the ticket	Contactless, Credit Card, Debit Card
Railcard	Whether the passenger is a National Railcard holder or not.	Adult, Disabled, Senior, None
Ticket Class	Seat class for the ticket	Standard, First Class
Ticket Type	When you bought or can use the ticket	Advance, Anytime, Off-peak
Price	Final cost of the ticket	43,23,3,13,76
Departure Station	Station to board the train	London Paddington, Liverpool Lime Street
Arrival Station	Station to exit the train	Birmingham New Street, London Euston
Departure	Time the train departed	2024-01-04 17:45
Scheduled Arrival	Time the train was scheduled to arrive at its destination	2024-01-04 19:05
Actual Arrival	Time the train arrived at its destination	2024-01-04 19:05
Journey Status	Whether the train was on time delayed or cancelled	Ontime, Delayed, Cancelled
Reason for Delay	Reason for the delay or cancellation	Staffing, Signal Failure Technical Failure, etc.
Refund Request	Whether the passenger requested a refund	Yes/No

Table1: *Description of fields in Maven Rail dataset.*

To analyze the quality of data, we can use Spiegelhalter's star rating (he defines data quality by giving a star rating from 0 to 4). I would give the data 3 stars as it is reasonably accurate (all the data are based on real stations) and well-constructed. While preparing the data for analysis, 5 columns were removed resulting in 13 columns. The CSV file did not need data wrangling which may correlate with accuracy of data, however, not reliably so.

EDA

Before conducting an EDA, the following steps were taken.

- The structure and summary of the data were observed.
- Dates were changed from character to POSIXct. POSIXct makes analysis on dates easier and enables you to address time zones.
- Columns with categorical variables were converted into factors. This was done to understand the impact of each category on other variables. For instance, converting Journey.Status helped us understand whether a specific status like 'Cancelled', led to the passenger asking for a refund. If we had grouped them, this distinction would not be possible.
- Null values were dealt with in the following manner. All null values were observed per column which showed that Actual.Arrival had the most number of NAs. Relating to the dataset, this makes sense. If the train arrived on its scheduled time, then the actual arrival will be NA. The important thing to notice was the NA rows in Departure and Scheduled.Arrival. There is no instance where Departure should be NA. Even for cancelled trains, Departure values were provided. There were some rows which had both Scheduled.Arrival and Actual.Arrival as NA which is also not possible as cancelled trains had data in at least one of these columns. Rows where Departure, and both Scheduled.Arrival and Actual.Arrival were NA were removed.

Since this dataset is predominantly categorical, EDA included chi-square tests and visualizations. A chi square test is used to compare categorical variables in large sample sizes to understand if the variables are independent of each other (i.e., there is no relation between them).

1. The first chi-square test was conducted on Journey.Status and Refund.Request. The values to focus on here are the X-squared value and the p-value. The X-squared value measures the difference between observed and expected values. The high number in this statistic indicates a strong association between the variables. The p-value indicates the probability of accepting or rejecting your null hypothesis. Ideally it should be less than 0.5 and as observed it is extremely small. This again shows good evidence of the two variables being strongly associated. To conclude, journey status affects whether the passenger requests a refund. This makes sense as passengers would most likely want a refund if their journey was delayed or canceled. A bar graph was plotted on these variables and as expected, most refunds were requested after the journey was cancelled.

Pearson's Chi-squared test

```
data: contingency_table  
X-squared = 7747, df = 2, p-value < 2.2e-16
```

Image 1: *Pearson's Chi Squared test for Journey.Status_and Refund.Request*

2. Another chi-square was conducted on Ticket.Type and Refund.Request and the following conclusions were drawn:
 - The two variables are independent of each other (p value <0.05).
 - The X-squared value was small (25.279) suggesting a weaker relation.
 - A bar graph plotted on these variables showed that tickets bought during Off-Peak had the highest refund rate followed by Advance and Anytime respectively.
 - An explanation for this behavior may include that people buying off-peak tickets don't expect their trains to be delayed or cancelled.
3. A chi-square conducted on Railcard and Refund.Request showed that these variables are independent of each other with a high X-squared value. Disabled and Senior citizens were less likely to ask for a refund as compared to adult and no railcard holders.

Other conclusions based on EDA:

- The most popular routes were measured using a barchart with the Manchester Piccadilly to Liverpool Lime Street being the most popular.
- Signal failure was the most frequent cause for train cancellations.
- Peak hours were between 6 to 7 AM and 5 to 6 PM which fits well with the normal working hours of 9 to 5.
- A heatmap on frequency of departures by hour and weekday showed that the departures on Tuesday and Wednesday were higher during PM timings. The hybrid mode of working adopted by many companies where employees are only asked to come in 2/3 days of the week might be an explanation.

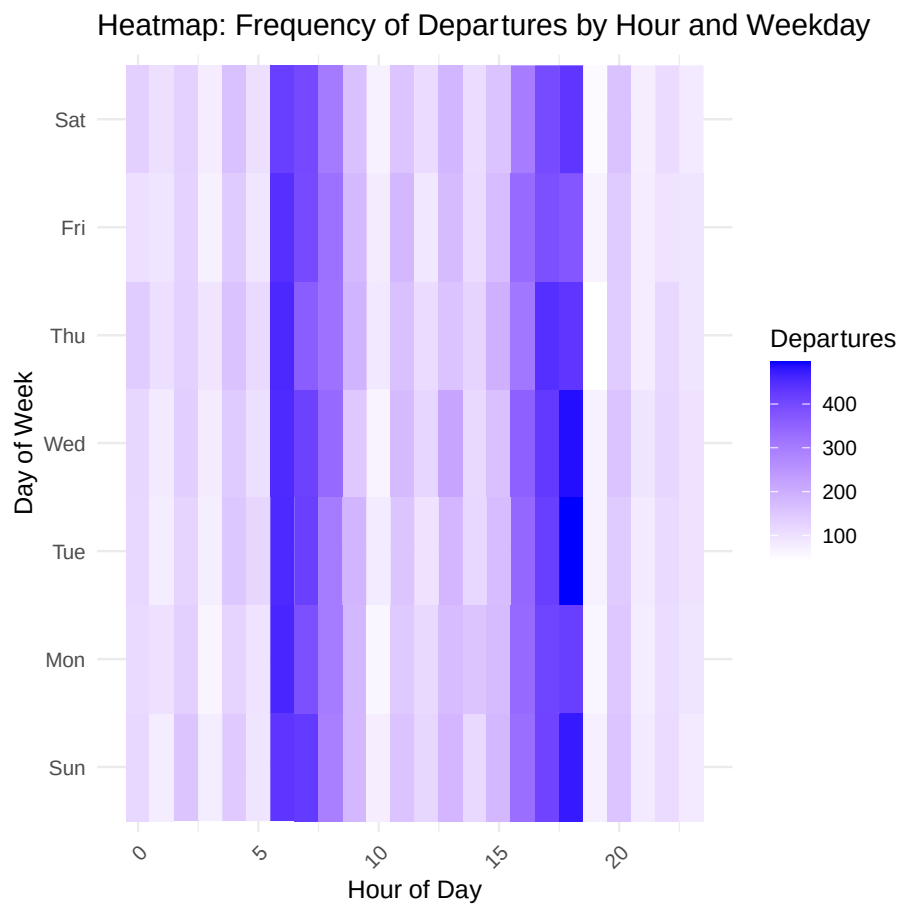


Image 2: *Heatmap for Frequency of Departures by Hour and Weekday*

Logistic Regression

A filtered dataset was created where Journey.Status was not On Time. A new column called MediumPrice was added where the price range was between 10 to 30 pounds. A logistic model was fitted which showed that MediumPrice has a significant impact on Refund.Request. This model was chosen because the target variable (Refund.Request) is binary.

LOGISTIC REGRESSION MODEL

2. Pr

$$\log\left(\frac{p}{1-p}\right) = \underbrace{\beta_0}_{\text{intercept}} + \underbrace{\beta_1}_{\text{coefficient}} \times \text{Medium Price}$$

1. Probability of refund after paying £5

$$p = \frac{1}{1+e^{-\theta}} \quad \text{where } \theta = \beta_0 + \beta_1 \times \text{Medium Price}$$

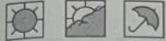
$$\theta = -1.07572 + 0.35440 \times \underbrace{0}_{\text{ticket price is not Medium}}$$

$$= -1.07572$$

$$p = \frac{1}{1+e^{1.07572}}$$

$$= \underline{0.252}$$

There is a 25% probability that a passenger with a £5 ticket will request a refund.



Mo Tu We Th Fi Sa Su

Memo No. _____

Date / /

2. Probability of refund after paying £25

$$\theta = -1.07572 + 0.35440 \times \underbrace{1}_{\text{ticket price is medium}}$$

$$\theta = -0.72132$$

$$p = \frac{1}{1 + e^{0.72132}}$$

$$= \underline{0.329}$$

There is a 32.9% probability that a passenger with a £25 ticket will request a refund.

Multivariate Regression

Four models were created.

1. In model 1, predictors include- Journey.Status and Reasons.For.Delay. The Weather factor in Reasons.For.Delay was kept as the reference category because it is an external factor that cannot be controlled, and therefore, acts neutral. All factors for Reasons.For.Delay were significant with Journey.StatusDelayed being significant and Journey.StatusOnTime being insignificant.
2. The second model used Journey.Status, Payment.Method, and Railcard as the predictors. Here, the Journey.StatusDelayed remained significant and Journey.StatusOnTime insignificant. All other factors were significant.
3. Model 3 used Journey.Status, Ticket.Type and Ticket.Class as predictors. Similar results were seen for Journey.Status with Ticket.ClassAnytime and off-peak being significant while Ticket.ClassStandard was insignificant.
4. Model 4 included only Journey.Status as a predictor and was used to predict the Refund.Request output in the ToPredict dataset. This was done because of the following reasons:
 - Journey status remained consistent across multiple models. This indicates the robustness of this variable and suggests that the relationship between Refund.Request and Journey.Status is not confounded by other predictors.
 - Using one predictor simplifies the prediction process. The conclusions drawn from the probabilities will be easier to communicate.
 - The consistency of Journey.Status may indicate that it'll generalize well with new data.
 - Practically speaking, the most direct relationship of Refund.Request is with Journey.Status.

The following probabilities were calculated.

- 30.4% probability that the passenger will request a refund when the journey status is Cancelled.
- 23.8% probability that the passenger will request a refund when the journey status is Delayed.
- 4.305023e-10 converts to an extremely small probability that the passenger will request a refund when the journey status is On Time.

APPENDIX

```
#loading all the required libraries
```

```
library(dplyr)
```

```
library(ggplot2)
```

```
library(lubridate)
```

```
#reading in the data
```

```
data<- read.csv(file = 'MavenRail.csv', header = TRUE, sep= ",")
```

```
data
```

```
#getting a gist of the data
```

```
summary(data)
```

```
> summary(data)
Payment.Method      Railcard      Ticket.Class      Ticket.Type      Price      Departure.Station
Length:31645      Length:31645      Length:31645      Length:31645      Min.   : 1.00      Length:31645
Class :character    Class :character    Class :character    Class :character    1st Qu.: 5.00      Class :character
Mode  :character    Mode  :character    Mode  :character    Mode  :character    Median :11.00      Mode  :character
                                   Mean   :23.43
                                   3rd Qu.:35.00
                                   Max.   :267.00

Arrival.Station     Departure      Scheduled.Arrival  Actual.Arrival     Journey.Status     Reason.for.Delay
Length:31645      Length:31645      Length:31645      Length:31645      Length:31645      Length:31645
Class :character    Class :character    Class :character    Class :character    Class :character    Class :character
Mode  :character    Mode  :character    Mode  :character    Mode  :character    Mode  :character    Mode  :character

Refund.Request
Length:31645
Class :character
Mode  :character
```

```
str(data)
```

```
colSums(is.na(data))
```



```
> str(data)
'data.frame': 31645 obs. of 13 variables:
 $ Payment.Method : chr "Contactless" "Credit Card" "Credit Card" "Credit Card" ...
 $ Railcard : chr "Adult" "Adult" "None" "None" ...
 $ Ticket.Class : chr "Standard" "Standard" "Standard" "Standard" ...
 $ Ticket.Type : chr "Advance" "Advance" "Advance" "Advance" ...
 $ Price : int 43 23 3 13 76 35 2 2 37 13 ...
 $ Departure.Station: chr "London Paddington" "London Kings Cross" "Liverpool Lime Street" "London Paddington" ...
 $ Arrival.Station : chr "Liverpool Lime Street" "York" "Manchester Piccadilly" "Reading" ...
 $ Departure : chr "2024-01-01 11:00" "2024-01-01 09:45" "2024-01-02 18:15" "2024-01-01 21:30" ...
 $ Scheduled.Arrival: chr "2024-01-01 13:30" "2024-01-01 11:35" "2024-01-02 18:45" "2024-01-01 22:30" ...
 $ Actual.Arrival : chr "2024-01-01 13:30" "2024-01-01 11:40" "2024-01-02 18:45" "2024-01-01 22:30" ...
 $ Journey.Status : chr "On Time" "Delayed" "On Time" "On Time" ...
 $ Reason.for.Delay : chr "" "Signal Failure" "" "" ...
 $ Refund.Request : chr "No" "No" "No" "No" ...

> colSums(is.na(data))
Payment.Method      Railcard      Ticket.Class      Ticket.Type      Price Departure.Station
           0              0              0              0              0              0
Arrival.Station      Departure Scheduled.Arrival Actual.Arrival Journey.Status Reason.for.Delay
           0              3              4             1880              0              0
Refund.Request
           0
```

#converting Departure, Scheduled Arrival, and Actual Arrival into POSIXct

```
data$Departure <- as.POSIXct( data$Departure, format="%Y-%m-%d %H:%M" )
```

```
data$Scheduled.Arrival <- as.POSIXct( data$Scheduled.Arrival, format="%Y-%m-%d
%H:%M" )
```

```
data$Actual.Arrival <- as.POSIXct( data$Actual.Arrival, format="%Y-%m-%d %H:%M" )
```

```
apply( data, class )
```

```
> apply( data, class )
$Payment.Method
[1] "character"

$Railcard
[1] "character"

$Ticket.Class
[1] "character"

$Ticket.Type
[1] "character"

$Price
[1] "integer"

$Departure.Station
[1] "character"

$Arrival.Station
[1] "character"

$Departure
[1] "POSIXct" "POSIXt"

$Scheduled.Arrival
[1] "POSIXct" "POSIXt"

$Actual.Arrival
[1] "POSIXct" "POSIXt"

$Journey.Status
[1] "factor"

$Reason.for.Delay
[1] "character"

$Refund.Request
[1] "character"
```

```

#converting columns into factors for easier analysis
data$Journey.Status <- as.factor(data$Journey.Status)
data$Refund.Request <- as.factor(data$Refund.Request)
data$Payment.Method <- as.factor(data$Payment.Method)
data$Railcard <- as.factor(data$Railcard)
data$Ticket.Class <- as.factor(data$Ticket.Class)
data$Ticket.Type <- as.factor(data$Ticket.Type)
data$Reason.for.Delay <- as.factor(data$Reason.for.Delay)

#removing rows where departure is NA
data <- data[!is.na(data$Departure), ]

#removing rows where both Scheduled.Arrival and Actual.Arrival are NA
data <- data[!(is.na(data$Scheduled.Arrival) & is.na(data$Actual.Arrival)), ]

```

#2 EDA

```

#Chi square test on Journey.Status and Refund.Request
contingency_table <- table(data$Journey.Status, data$Refund.Request)
chi_square_test <- chisq.test(contingency_table)
print(chi_square_test)

```

```
> print(chi_square_test)
```

Pearson's Chi-squared test

```

data: contingency_table
X-squared = 7747, df = 2, p-value < 2.2e-16

```

```

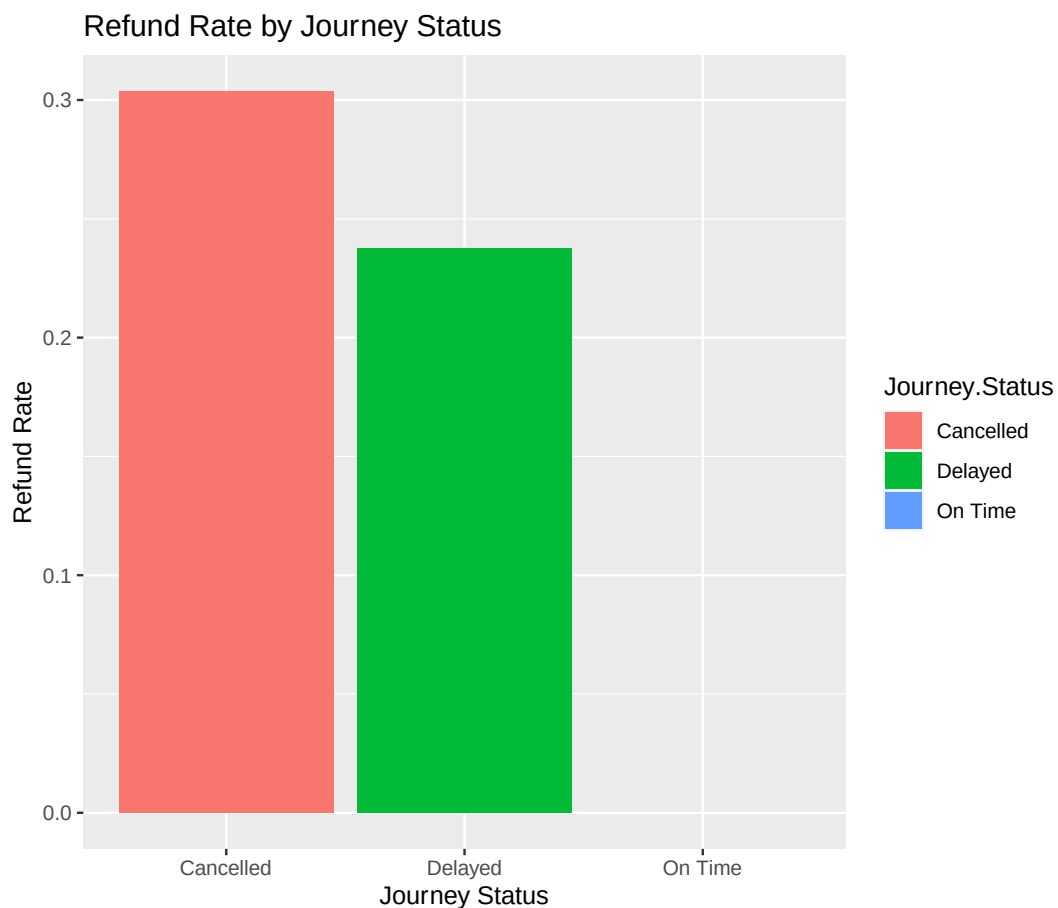
#Bar Graph
refunds_by_status <- data %>%
  group_by(Journey.Status) %>%
  summarise(Refund.Count = sum(as.numeric(Refund.Request == "Yes"), na.rm = TRUE),

```

```
Total.Count = n()) %>%
mutate(Refund.Rate = Refund.Count / Total.Count)
```

	Journey.Status	Refund.Count	Total.Count	Refund.Rate
1	Cancelled	570	1877	0.3036761
2	Delayed	544	2289	0.2376584
3	On Time	0	27474	0.0000000

```
ggplot(refunds_by_status, aes(x = Journey.Status, y = Refund.Rate, fill = Journey.Status)) +
  geom_bar(stat = "identity") +
  labs(title = "Refund Rate by Journey Status", y = "Refund Rate", x = "Journey Status")
```



```
#Chi square on Ticket.Type and Refund.Request
contingency_table1 <- table(data$Ticket.Type, data$Refund.Request)
```

```
chi_square_test1 <- chisq.test(contingency_table1)
print(chi_square_test1)
```

```
> print(chi_square_test1)
```

Pearson's Chi-squared test

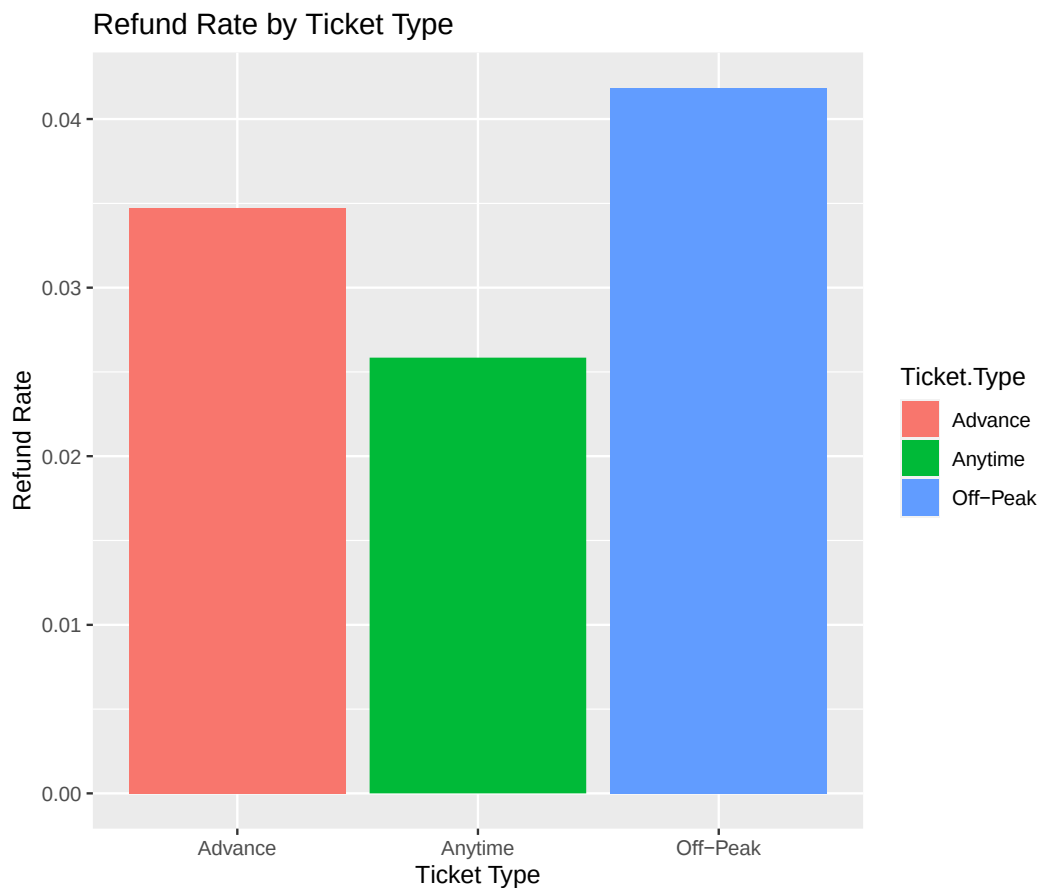
```
data: contingency_table1
X-squared = 25.279, df = 2, p-value = 3.241e-06
```

#Bar Graph

```
refunds_by_ticket <- data %>%
  group_by(Ticket.Type) %>%
  summarise(Refund.Count = sum(as.numeric(Refund.Request == "Yes"), na.rm = TRUE),
            Total.Count = n()) %>%
  mutate(Refund.Rate = Refund.Count / Total.Count)
```

	Ticket.Type	Refund.Count	Total.Count	Refund.Rate
1	Advance	610	17556	0.03474596
2	Anytime	138	5340	0.02584270
3	Off-Peak	366	8744	0.04185727

```
ggplot(refunds_by_ticket, aes(x = Ticket.Type, y = Refund.Rate, fill = Ticket.Type)) +
  geom_bar(stat = "identity") +
  labs(title = "Refund Rate by Ticket Type", y = "Refund Rate", x = "Ticket Type")
```



#Chi-Square Test for Railcard and Refund Request

```
railcard_refund_table <- table(data$Railcard, data$Refund.Request)
```

```
chi_square_railcard <- chisq.test(railcard_refund_table)
```

```
chi_square_railcard
```

```
> chi_square_railcard
```

Pearson's Chi-squared test

```
data: railcard_refund_table
```

```
X-squared = 482.44, df = 3, p-value < 2.2e-16
```

#Bar graph

```
ggplot(data, aes(x = Railcard, fill = Refund.Request)) +
```

```
geom_bar(position = "dodge") +
```

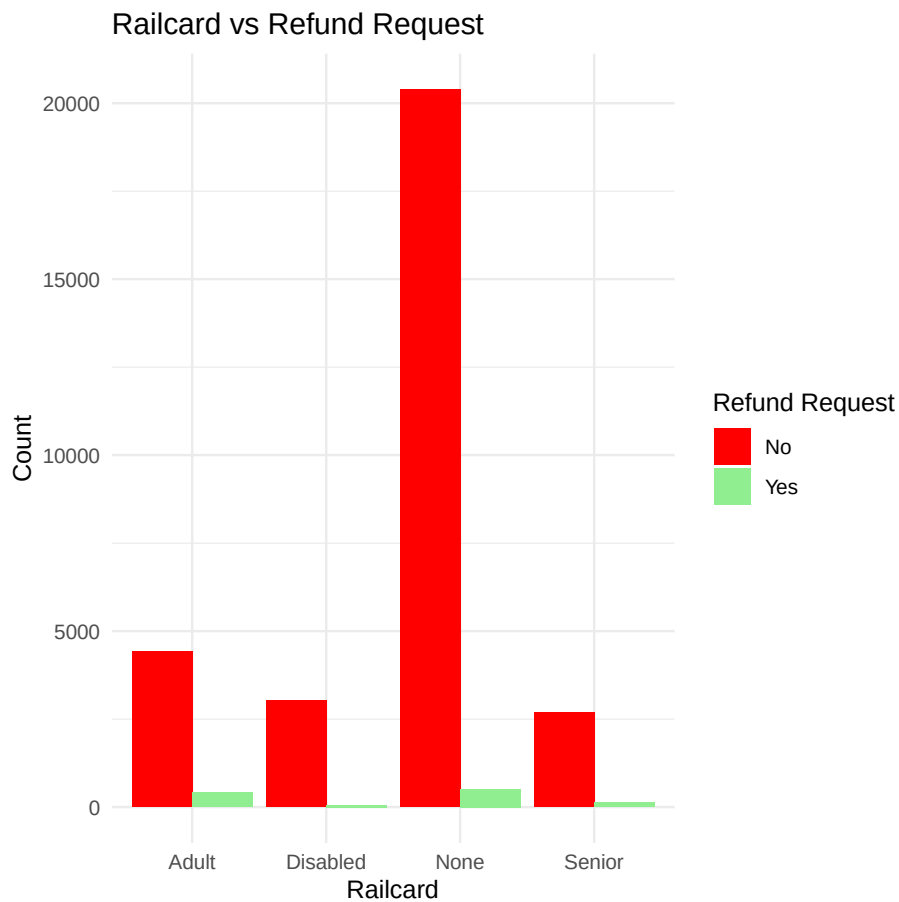
```
labs(title = "Railcard vs Refund Request",
```

```
  x = "Railcard",
```

```

y = "Count",
fill = "Refund Request") +
scale_fill_manual(values = c("red", "lightgreen")) +
theme_minimal()

```



#Most and least popular routes

```

routes <- data %>%
  group_by(Departure.Station, Arrival.Station) %>%
  summarise(Count = n()) %>%
  arrange(desc(Count))

```

```

top_routes <- head(routes, 10)
bottom_routes <- tail(routes, 10)

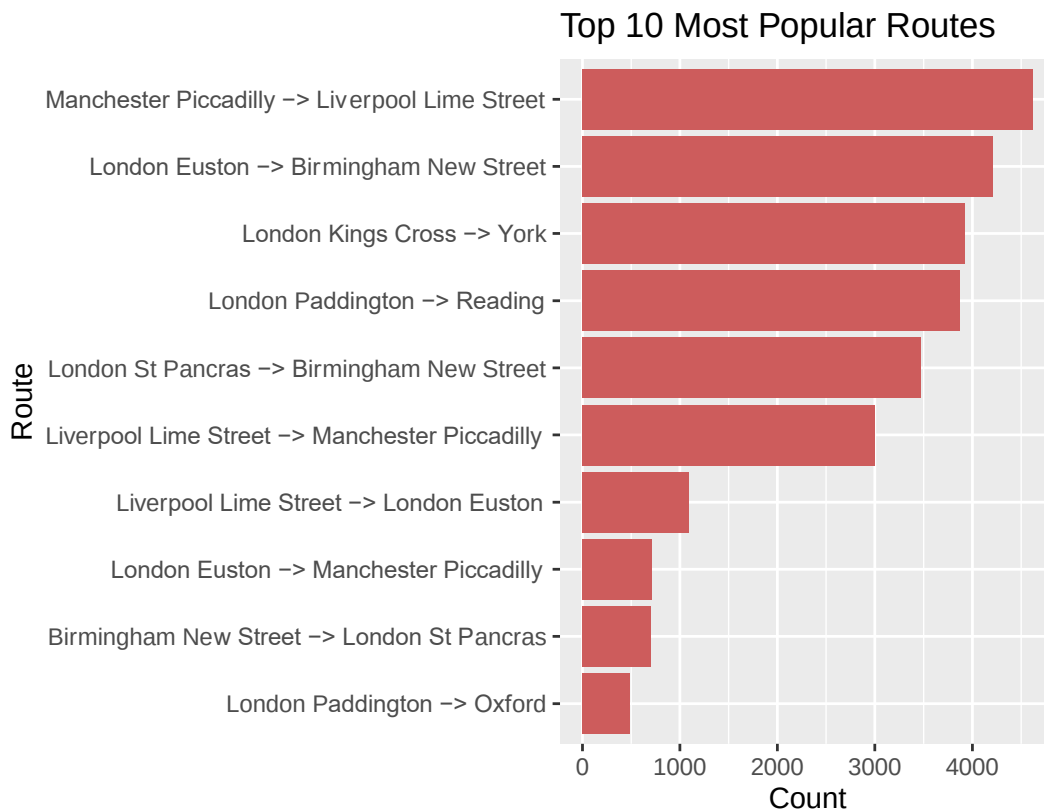
```

```

ggplot(top_routes, aes(x = reorder(paste(Departure.Station, Arrival.Station, sep = " -> "),
Count), y = Count)) +

```

```
geom_bar(stat = "identity", fill = "indianred") +
coord_flip() +
labs(title = "Top 10 Most Popular Routes", y = "Count", x = "Route")
```



```
#Frequency of causes for Cancelled Journey.Status
cancellations <- data %>%
filter(Journey.Status == "Cancelled") %>%
count(Reason.for.Delay, sort = TRUE)
```

	Reason.for.Delay	n
1	Signal Failure	518
2	Weather	445
3	Staffing	237
4	Technical Issue	234
5	Traffic	227
6	Staff	216

```
ggplot(cancellations, aes(x = reorder(Reason.for.Delay, n), y = n, fill = Reason.for.Delay)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(title = "Main Causes of Train Cancellations", y = "Count", x = "Reason for Delay")
```



#understanding peak hours

#extracting hours and coding it as AM/PM

```
data$Hour <- format(data$Departure, "%H")
```

```
data$Hour <- as.numeric(data$Hour)
```

```
data$TimeOfDay <- ifelse(data$Hour < 12, "AM", "PM")
```

```
ggplot(data, aes(x = Hour, fill = TimeOfDay)) +
```

```
  geom_histogram(binwidth = 1, position = "dodge", color = "black") +
```

```
  scale_x_continuous(breaks = 0:23) + # Set breaks for 24 hours
```

```
  labs(title = "Peak Hours of Train Journeys",
```

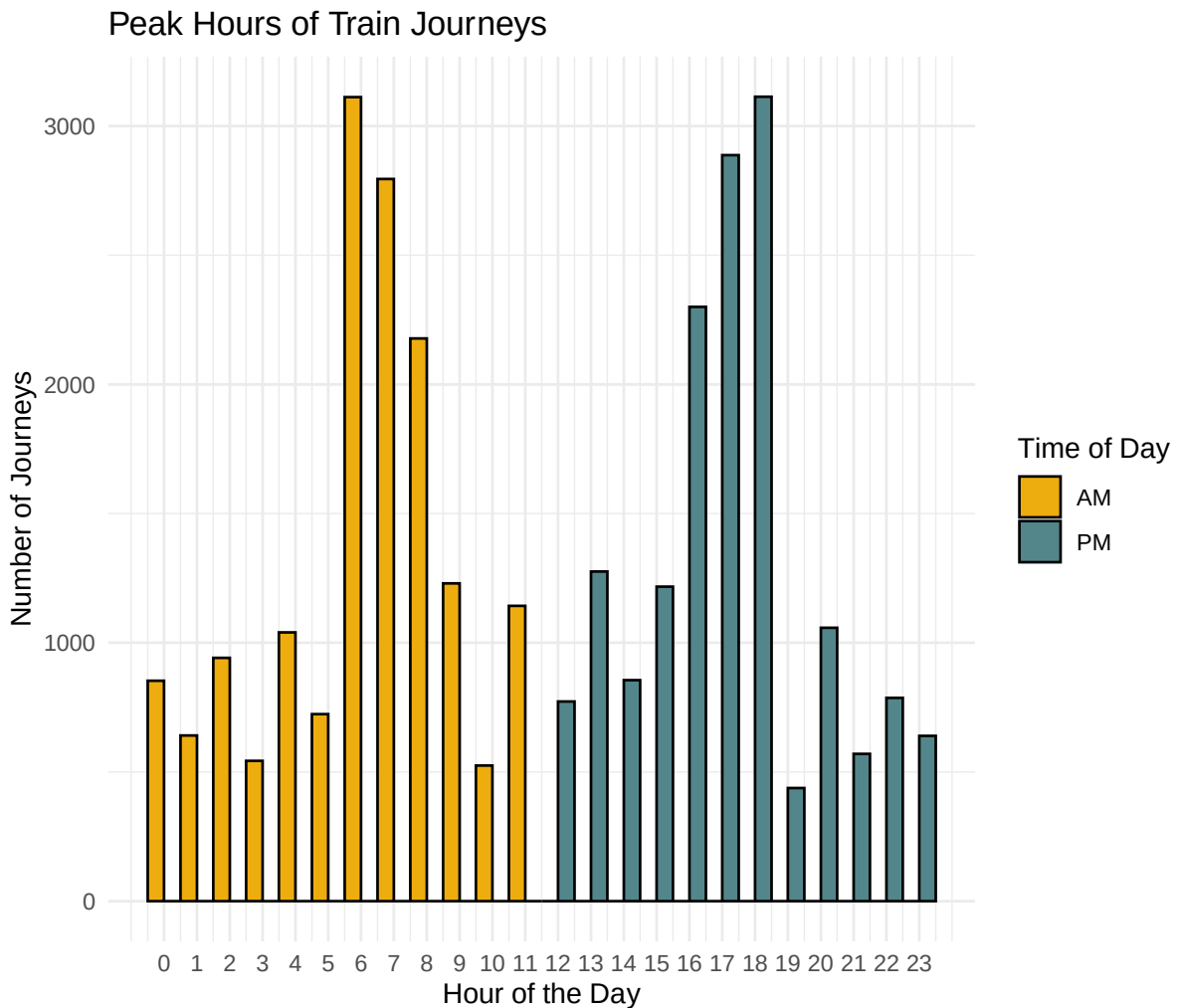
```
        x = "Hour of the Day",
```



```

y = "Number of Journeys",
fill = "Time of Day") +
theme_minimal() +
scale_fill_manual(values = c("AM" = "darkgoldenrod2", "PM" = "cadetblue4"))

```



```
#Heatmap for frequency of departures
```

```
#making new columns for hours and weekdays
```

```
data$Weekday <- wday(data$Departure, label = TRUE)
```

```
#Counting the number of departures for each combination of hour and weekday
```

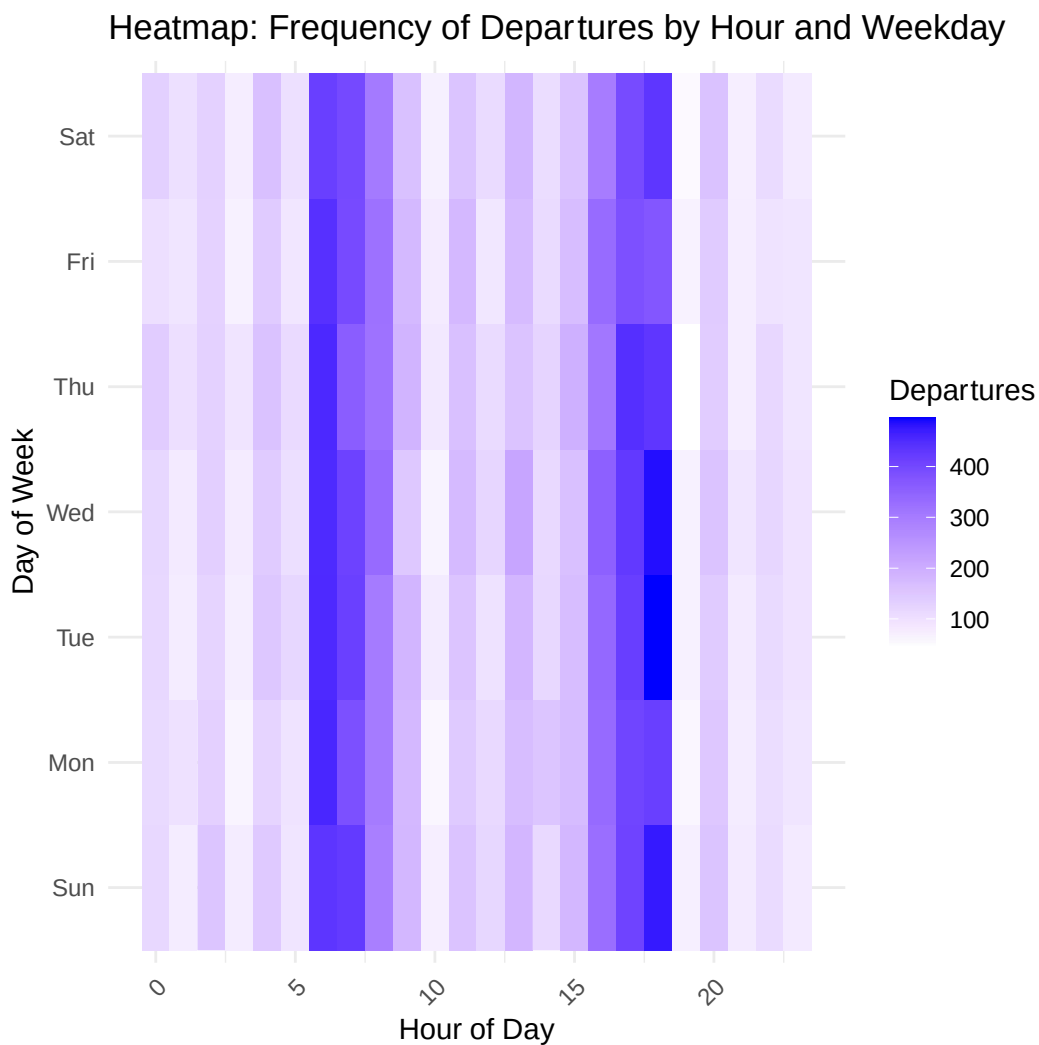
```
departure_counts_heatmap <- data %>%
```

```
  group_by(Weekday, Hour) %>%
```

```
  summarise(Departures = n()) %>%
```

```
  ungroup()
```

```
#Creating a heatmap for frequency of departures by hour and weekday
ggplot(departure_counts_heatmap, aes(x = Hour, y = Weekday, fill = Departures)) +
  geom_tile() +
  scale_fill_gradient(low = "white", high = "blue") +
  labs(title = "Heatmap: Frequency of Departures by Hour and Weekday",
       x = "Hour of Day",
       y = "Day of Week",
       fill = "Departures") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



#3 Adding the DelayInMinutes column

```
data$DelayInMinutes <- ifelse(
  data$Journey.Status == "Delayed",
  as.numeric(difftime(data$Actual.Arrival, data$Scheduled.Arrival, units = "mins")),
  NA
)
```

#4 Focusing on rows where Journey.Status is not "On Time"

```
filtered_df <- data[data$Journey.Status != "On Time", ]
```

#Adding the MediumPrice column

```
filtered_df$MediumPrice <- ifelse(filtered_df$Price > 10 & filtered_df$Price <= 30, TRUE,
FALSE)
head(filtered_df)
```

```
> head(filtered_df)
```

	Payment.Method	Railcard	Ticket.Class	Ticket.Type	Price	Departure.Station	Arrival.Station		
2	Credit Card	Adult	Standard	Advance	23	London Kings Cross	York		
9	Credit Card	None	Standard	Advance	37	London Euston	York		
21	Debit Card	Adult	Standard	Advance	7	Birmingham New Street	Manchester Piccadilly		
27	Credit Card	Senior	First Class	Advance	34	Oxford	Bristol Temple Meads		
40	Credit Card	None	Standard	Advance	7	London Euston	Birmingham New Street		
46	Credit Card	None	First Class	Advance	134	Manchester Piccadilly	London Euston		

	Departure	Scheduled.Arrival	Actual.Arrival	Journey.Status	Reason.for.Delay	Refund.Request	Hour
2	2024-01-01 09:45:00	2024-01-01 11:35:00	2024-01-01 11:40:00	Delayed	Signal Failure	No	9
9	2024-01-01 00:00:00	2024-01-01 01:50:00	2024-01-01 02:07:00	Delayed	Signal Failure	No	0
21	2024-01-01 11:15:00	2024-01-01 12:35:00	2024-01-01 13:06:00	Delayed	Technical Issue	Yes	11
27	2024-01-01 14:15:00	2024-01-01 15:30:00	2024-01-01 15:54:00	Delayed	Signal Failure	Yes	14
40	2024-01-02 02:15:00	2024-01-02 03:35:00	<NA>	Cancelled	Technical Issue	No	2
46	2024-01-02 03:30:00	2024-01-02 05:20:00	2024-01-02 05:31:00	Delayed	Weather	No	3

	TimeOfDay	Weekday	DelayInMinutes	MediumPrice
2	AM	Mon	5	TRUE
9	AM	Mon	17	FALSE
21	AM	Mon	31	FALSE
27	PM	Mon	24	FALSE
40	AM	Tue	NA	FALSE
46	AM	Tue	11	FALSE

#MODEL (logistic regression)

```
model <- glm(Refund.Request ~ MediumPrice, data = filtered_df, family = binomial)
```

```
summary(model)
```

```
> summary(model)
```

Call:

```
glm(formula = Refund.Request ~ MediumPrice, family = binomial,  
     data = filtered_df)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.07572	0.03928	-27.383	< 2e-16 ***
MediumPriceTRUE	0.35440	0.08723	4.063	4.85e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4838 on 4165 degrees of freedom
Residual deviance: 4822 on 4164 degrees of freedom
AIC: 4826

Number of Fisher Scoring iterations: 4

#5

#MODEL 1

#keeping weather as the reference category

```
data$Reason.for.Delay <- relevel(data$Reason.for.Delay, ref = "Weather")
```

```
model1 <- glm(Refund.Request ~ Journey.Status + Reason.for.Delay,  
              data = data, family = binomial)
```

```
summary(model1)
```

```
> summary(model1)
```

Call:

```
glm(formula = Refund.Request ~ Journey.Status + Reason.for.Delay,  
     family = binomial, data = data)
```

Coefficients: (1 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-2.0347	0.1036	-19.635	< 2e-16	***
Journey.StatusDelayed	-0.3039	0.0789	-3.852	0.000117	***
Journey.StatusOn Time	-19.5314	176.3616	-0.111	0.911818	
Reason.for.Delay	NA	NA	NA	NA	
Reason.for.DelaySignal Failure	0.9162	0.1208	7.585	3.32e-14	***
Reason.for.DelayStaff	0.7683	0.1564	4.913	8.95e-07	***
Reason.for.DelayStaffing	1.9182	0.1366	14.046	< 2e-16	***
Reason.for.DelayTechnical Issue	2.4323	0.1188	20.477	< 2e-16	***
Reason.for.DelayTraffic	1.6497	0.1505	10.962	< 2e-16	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 9644.2 on 31639 degrees of freedom
Residual deviance: 4230.0 on 31632 degrees of freedom
AIC: 4246

Number of Fisher Scoring iterations: 20

#MODEL 2

```
model2 <- glm(Refund.Request ~ Journey.Status + Payment.Method + Railcard,  
              data = data, family = binomial)  
summary(model2)
```

```
> summary(model2)
```

Call:

```
glm(formula = Refund.Request ~ Journey.Status + Payment.Method +  
     Railcard, family = binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-0.1051	0.1181	-0.890	0.373572
Journey.StatusDelayed	-1.8260	0.1115	-16.382	< 2e-16 ***
Journey.StatusOn Time	-22.3518	270.8977	-0.083	0.934241
Payment.MethodCredit Card	-0.2060	0.1017	-2.026	0.042792 *
Payment.MethodDebit Card	3.5276	0.1568	22.501	< 2e-16 ***
RailcardDisabled	-1.8590	0.1880	-9.891	< 2e-16 ***
RailcardNone	-0.7433	0.1098	-6.771	1.28e-11 ***
RailcardSenior	-0.5803	0.1728	-3.358	0.000785 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 9644.2 on 31639 degrees of freedom
Residual deviance: 3519.8 on 31632 degrees of freedom
AIC: 3535.8

Number of Fisher Scoring iterations: 21

#MODEL 3

```
model3 <- glm(Refund.Request ~ Journey.Status + Ticket.Type + Ticket.Class,  
              data = data, family = binomial)
```

```
summary(model3)
```

```
> summary(model3)
```

Call:

```
glm(formula = Refund.Request ~ Journey.Status + Ticket.Type +  
     Ticket.Class, family = binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-0.78799	0.12095	-6.515	7.27e-11	***
Journey.StatusDelayed	-0.33500	0.07093	-4.723	2.33e-06	***
Journey.StatusOn Time	-20.77936	175.45915	-0.118	0.906	
Ticket.TypeAnytime	-0.64486	0.10462	-6.164	7.11e-10	***
Ticket.TypeOff-Peak	0.35043	0.08037	4.360	1.30e-05	***
Ticket.ClassStandard	-0.03029	0.11767	-0.257	0.797	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 9644.2 on 31639 degrees of freedom
Residual deviance: 4732.2 on 31634 degrees of freedom
AIC: 4744.2

Number of Fisher Scoring iterations: 20

#MODEL 4

```
model4 <- glm(Refund.Request ~ Journey.Status,  
              data = data, family = binomial)
```

```
summary(model4)
```

```

> summary(model4)

Call:
glm(formula = Refund.Request ~ Journey.Status, family = binomial,
    data = data)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)    -0.82985    0.05019 -16.533  < 2e-16 ***
Journey.StatusDelayed -0.33571    0.07022  -4.781 1.75e-06 ***
Journey.StatusOn Time -20.73622   176.36163  -0.118   0.906
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 9644.2  on 31639  degrees of freedom
Residual deviance: 4815.2  on 31637  degrees of freedom
AIC: 4821.2

Number of Fisher Scoring iterations: 20

```

#Using models with the ToPredict dataset

```
data1<- read.csv(file = 'ToPredict.csv', header = TRUE, sep= ",")
```

data1

```

> data1

```

	Payment.Method	Railcard	Ticket.Class	Ticket.Type	Price	Departure.Station	Arrival.Station	Departure
1	Debit Card	None	First Class	Advance	54	London St Pancras	Birmingham New Street	2024-01-04 17:45
2	Credit Card	None	Standard	Advance	7	London Euston	Birmingham New Street	2024-01-05 08:15
3	Debit Card	None	Standard	Off-Peak	113	Liverpool Lime Street	London Euston	2024-01-09 15:30
4	Contactless	Adult	Standard	Off-Peak	3	Liverpool Lime Street	Manchester Piccadilly	2024-01-31 05:45
5	Credit Card	None	Standard	Off-Peak	4	Manchester Piccadilly	Liverpool Lime Street	2024-02-10 16:00
6	Contactless	None	Standard	Advance	3	Manchester Piccadilly	Liverpool Lime Street	2024-02-25 15:45
7	Debit Card	None	Standard	Off-Peak	126	Manchester Piccadilly	London Euston	2024-03-20 15:30
8	Credit Card	None	Standard	Advance	22	Birmingham New Street	London St Pancras	2024-04-16 04:30
	Scheduled.Arrival	Actual.Arrival	Journey.Status	Reason.for.Delay				
1	2024-01-04 19:05	2024-01-04 19:05	On Time					
2	2024-01-05 09:35	2024-01-05 09:35	On Time					
3	2024-01-09 17:45	2024-01-09 18:07	Delayed	Staffing				
4	2024-01-31 06:15	2024-01-31 06:49	Delayed	Signal Failure				
5	2024-02-10 16:30	<NA>	Cancelled	Technical Issue				
6	2024-02-25 16:15	<NA>	Cancelled	Staffing				
7	2024-03-20 17:20	2024-03-20 17:36	Delayed	Staffing				
8	2024-04-16 05:50	<NA>	Cancelled	Signal Failure				

#adding probabilities as a new column

```
data1$Refund_Probability <- predict(model4, newdata = data1, type = "response")
```


Journey.Status ▾	Reason.for.Delay ▾	Refund_Probability ▾
On Time		4.305023e-10
On Time		4.305023e-10
Delayed	Staffing	2.376584e-01
Delayed	Signal Failure	2.376584e-01
Cancelled	Technical Issue	3.036761e-01
Cancelled	Staffing	3.036761e-01
Delayed	Staffing	2.376584e-01
Cancelled	Signal Failure	3.036761e-01